

TP2 Java Yassine Hamrouni

Exercice 1 - Assignation, égalité, référence

1- Quel est le type de s ? Comment le compilateur fait-il pour savoir qu'il existe une méthode length() sur s ?

Le type de s est String. Dans le code Java, quand on utilise var pour déclarer une variable et qu'on lui assigne directement une valeur, le compilateur déduit le type de la variable à partir de cette valeur. Ici, puisque s est assignée la valeur "toto", qui est clairement une chaîne de caractères (entourée de guillemets), le compilateur déduit que s est de type String.

Le type String en Java est une classe qui fournit de nombreuses méthodes pour manipuler des chaînes de caractères. L'une de ces méthodes est length(), qui retourne le nombre de caractères dans la chaîne. Le compilateur sait qu'une méthode length() existe sur s parce que s est identifiée comme une instance de String, et length() est une méthode définie dans la classe String.

2- Qu'affiche le code suivant ?

```
var s1 = "toto";  
var s2 = s1;  
var s3 = new String(s1);  
  
System.out.println(s1 == s2);  
System.out.println(s1 == s3);
```

Ce code affiche:

true

false

`s1 == s2` retourne true parce que `s2` est assigné à `s1`, ce qui signifie qu'ils référencent exactement le même objet dans la mémoire. En Java, l'opérateur `==` compare les références (les adresses en mémoire) pour les objets, et non pas leur contenu.

`s1 == s3` retourne false car `s3` est créé avec `new String(s1)`, ce qui force la création d'un nouvel objet String dans la mémoire, même si le contenu est identique à celui de `s1`. Ainsi, `s1` et `s3` ont le même contenu mais ne référencent pas le même objet en mémoire.

3- Quelle est la méthode à utiliser si l'on veut tester si le contenu des chaînes de caractères est le même ?

Pour tester si le contenu de deux chaînes de caractères est le même, on doit utiliser la méthode `equals()`, ainsi :

```
var s4 = "toto";
var s5 = new String(s4);

System.out.println(s4.equals(s5));
```

Cette méthode compare le contenu des deux chaînes de caractères et retournera true si les contenus sont identiques, peu importe si les deux objets String sont stockés à des emplacements différents en mémoire.

4- Qu'affiche le code suivant ?

```
var s6 = "toto";
var s7 = "toto";

System.out.println(s6 == s7);
```

Ce code affiche true.

En Java, les littéraux de chaînes (c'est-à-dire, les chaînes de caractères définies directement dans le code en utilisant des guillemets) sont internés. Cela signifie que lorsqu'on crée deux chaînes de caractères identiques de cette manière, Java ne crée pas un nouvel objet pour chaque chaîne. Au lieu de cela, les deux variables `s6` et `s7` pointent vers le même objet dans le pool de chaînes de caractères de la JVM. Par conséquent, `s6 == s7` retourne true car les deux variables référencent le même objet en mémoire.

5- Expliquer pourquoi il est important que `java.lang.String` ne soit pas mutable.

La classe `String` en Java est immuable, ce qui signifie qu'une fois qu'une instance de `String` est créée, son contenu ne peut pas être modifié. Cette immutabilité a plusieurs avantages :

La classe `String` en Java est immuable, ce qui signifie qu'une fois une instance de `String` créée, son contenu ne peut pas être modifié. Cette caractéristique est importante pour plusieurs raisons :

Sécurité : La mutabilité des chaînes pourrait conduire à des vulnérabilités de sécurité, où le contenu d'une chaîne pourrait être modifié de manière inattendue.

Synchronisation : L'immutabilité rend les objets `String` naturellement thread-safe, ce qui signifie qu'ils peuvent être partagés entre plusieurs threads sans nécessiter de synchronisation explicite.

Optimisation : L'immutabilité permet l'internement des chaînes, ce qui réduit la consommation de mémoire et améliore les performances en réutilisant les instances de chaînes identiques.

6- Qu'affiche le code suivant ?

```
var s8 = "hello";  
s8.toUpperCase();  
System.out.println(s8);
```

La méthode `toUpperCase()` crée une nouvelle chaîne de caractères où tous les caractères de la chaîne originale sont convertis en majuscules. Cependant, puisque les objets `String` en Java sont immutables, l'appel à `toUpperCase()` ne modifie pas l'objet `s8` lui-même mais retourne plutôt une nouvelle chaîne de caractères. Dans cet exemple, le résultat de `s8.toUpperCase()` n'est pas assigné à une variable, donc la chaîne originale `s8` reste inchangée, et `hello` est affiché.

Exercice 2 - En morse. Stop.

1- A quoi sert l'objet `java.lang.StringBuilder` ?

`java.lang.StringBuilder` est utilisé pour créer une chaîne de caractères mutable. Contrairement aux objets `String` qui sont immuables, `StringBuilder` permet de modifier la chaîne de caractères sans créer à chaque fois un nouvel objet, ce qui le rend plus performant pour les opérations répétitives de concaténation.

2- Pourquoi sa méthode `append(String)` renvoie-t-elle un objet de type `StringBuilder` ?

La méthode `append(String)` de `StringBuilder` renvoie un objet de type `StringBuilder` pour permettre le chaînage des appels à `append`. Cela permet d'ajouter plusieurs chaînes en une seule instruction, rendant le code plus lisible et concis.

3- Quel est l'avantage par rapport à la solution précédente ?

L'avantage d'utiliser `StringBuilder` est que cela réduit le coût de performance, car les chaînes de caractères ne sont pas recréées à chaque concaténation, contrairement à l'opérateur `+`

4- Pourquoi peut-on utiliser `' '` à la place de `" "` ?

Dans le code fourni, l'utilisation de `' '` (caractère espace) au lieu de `" "` (chaîne contenant un espace) est une optimisation légère car elle évite de créer un objet `String` pour un seul caractère. En Java, les caractères entre guillemets simples sont des `char`, qui sont plus légers à manipuler que les objets `String`.

Compiler le code puis utiliser la commande `javap` pour afficher le bytecode Java (qui n'est pas un assembleur) généré. Que pouvez-vous en déduire ?

`StringBuilder` génère un code plus optimisé pour la concaténation de chaînes.

5- Compiler le code de la question 1, puis utiliser la commande `javap` pour afficher le bytecode Java généré. Que pouvez-vous en déduire ?

En comparant le bytecode des versions, `StringBuilder` génère un code plus optimisé pour la concaténation de chaînes.

Quand utiliser `StringBuilder.append()` plutôt que `+` ?

Utiliser `StringBuilder.append()` plutôt que l'opérateur `+` dans les situations où de nombreuses concaténations sont effectuées, particulièrement dans les boucles. Cela améliore la performance en réduisant la quantité de créations d'objets `String` intermédiaires.

Dans quel cas doit-on utiliser `StringBuilder.append()` plutôt que le `+` ?

On doit utiliser `StringBuilder.append()` plutôt que l'opérateur `+` pour la concaténation de chaînes de caractères quand on effectue de nom-

breuses concaténations, surtout dans les boucles, car cela améliore la performance et l'efficacité en termes de mémoire.

Et pourquoi est-ce que le chargé de TD va me faire les gros yeux si j'écris un `+` dans un appel à la méthode `append`?

Utiliser `+` dans un appel à la méthode `append()` annule les avantages de performance de `StringBuilder`, car cela force la création d'une chaîne intermédiaire avant d'appeler `append()`. C'est pour cette raison que cela pourrait attirer un regard désapprobateur de la part de votre chargé de TD, car cela indique une mécompréhension de l'optimisation que `StringBuilder` est censé apporter

Exercice 3 - Reconnaissance de motifs

1- A quoi servent la classe `java.util.regex.Pattern` et sa méthode `compile` ?

La classe `java.util.regex.Pattern` et sa méthode `compile` servent à définir une expression régulière et à la compiler en une représentation optimisée pour des opérations de recherche, de correspondance, et de manipulation de chaînes. La méthode `compile` transforme la chaîne d'expression régulière en une instance de `Pattern` qui peut ensuite être utilisée pour créer un objet `Matcher`. Ce processus de compilation améliore les performances lors de l'utilisation répétée de l'expression régulière.

A quoi sert la classe `java.util.regex.Matcher` ?

La classe `java.util.regex.Matcher` est utilisée pour effectuer des opérations de correspondance entre une expression régulière compilée (représentée par une instance de `Pattern`) et une séquence de caractères. Elle fournit des méthodes pour différentes tâches telles que la recherche de correspondances, le remplacement de texte, et l'extraction de sous-chaînes qui correspondent à des parties spécifiques de l'expression régulière. Le `Matcher` est capable de trouver plusieurs correspondances dans une même séquence de caractères et permet d'accéder à chacune d'elles, de les modifier ou de les utiliser pour d'autres traitements.

4- Modifiez le programme pour ne conserver que les liens contenant le mot `java` ou `Java`

```
if(url.toLowerCase().contains("java")) {  
    System.out.println("==>" + url);  
}
```